

# Build Task — Project 4 as an API — Hints & Solution

---

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper FastAPI/SQLite/Docker), **Advanced** (how a real production service actually handles this). Read Basic first even if you've done every Practice Exercise above — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — What you're building, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)
- [Solution](#)

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Hint 1 — What you're building, and the exact pieces

### Basic Version

You're combining everything from the Practice Exercises above into one working service: a FastAPI route that runs Project 4's multi-agent task, a SQLite table that remembers every run, log lines tagged with which request they belong to, a Dockerfile that packages the whole thing, and a simple limit on how often one caller can hit the route.

Nothing here is a new idea — it's `chat_route` (a route wrapping real logic) plus `sqlite_persistence` (saving and reading back runs) plus `db_failure_and_docker` (a real Dockerfile) plus one new small piece: rate limiting.

The exact pieces:

- `POST /run-task` — a route wrapping Project 4's task-running function, same shape as `chat_route`'s `/chat`.
- A `runs` table with `id`, `input`, `output`, `log`, `status`, `created_at` — same shape as `sqlite_persistence`'s table, plus `log` and `status` columns.
- `GET /runs/{id}` — look up one run by ID, same as `sqlite_persistence`'s Advanced version.
- A request ID: a random short string generated once per request, included in every log line for that request.
- A dictionary counting recent requests per caller, to reject a caller going over some limit with a `429` status code.

### Intermediate Version

Think of the whole Build Task as 3 layers stacked on top of each other, each one you've already built separately: the **API layer** ( `chat_route` 's pattern — typed request in, typed response out, 4xx/5xx handled correctly), the **storage layer** ( `sqlite_persistence` 's pattern — but now recording a `status` that changes as the run progresses: `"started"` when the row is first written, `"completed"` or `"failed"` once it's done), and the **operations layer** ( `db_failure_and_docker` 's pattern — a Dockerfile, secrets only via env vars — plus the new pieces: per-request log tagging, and rate limiting).

**Why status needs 3 states, not just "done":** unlike `sqlite_persistence` 's single-step insert, a task run can fail partway through — the agent might crash, an API call might time out. Writing a `"started"` row *before* running the task, then updating it to `"completed"` or `"failed"` after,

means a crashed run still leaves a real, honest record instead of either nothing (if you only write at the end) or a silently wrong "success" row.

**Why the request ID matters here specifically:** a multi-agent run produces many log lines across several steps (planning, tool calls, sub-agent handoffs). Without a shared ID on every one of those lines, you can't tell which log lines belong to which HTTP request once two people are calling your API at the same time — this is exactly `chat_route` Advanced's request-ID middleware, applied to a run that produces far more log output per request.

The exact pieces:

- `import uuid; request_id = str(uuid.uuid4())[:8]` — generated once per request, in middleware, stored on `request.state.request_id`.
- `logging.Formatter` with `%(request_id)s` in the format string, or `extra={"request_id": request_id}` passed to every `logger.info(...)` call for that request.
- A `Run` table: `id, input, output, log, status, created_at` — `log` can just be a text column holding the agent's step-by-step trace as one string (join it with `\n`, or store JSON).
- A plain-dictionary rate limiter: `request_counts: dict[str, list[float]] = {}`, storing timestamps per caller (by IP, or an API key if you have one), and rejecting once too many timestamps fall inside your time window.
- `raise HTTPException(status_code=429, detail="Rate limit exceeded")`.

## Advanced Version

Three real design questions the Requirements don't spell out, that a production version has to answer:

**Does the route block until the whole multi-agent run finishes, or return immediately and let the client poll?** A multi-agent task can take much longer than a typical HTTP request should — if `POST /run-task` blocks synchronously for 30+ seconds, you tie up a worker thread the whole time, and a client's own HTTP client might time out waiting. The two real options: keep it synchronous for this Build Task (simplest, matches "same shape as Project 4's terminal input, now over HTTP" from the Requirements) and note the limitation, or return immediately with a `"started"` status and `id`, run the task with `BackgroundTasks` (or a real task queue, which Doc12's "Advanced" topic tier mentions), and let the client poll `GET /runs/{id}` until `status` becomes `"completed"`.

**What's the actual, honest limitation of an in-memory rate limiter — and why does the Requirements section ask you to write it down?** A plain Python dictionary living in your process's memory forgets everything the moment the process restarts, and — more importantly — if you ever run more than one instance/worker of this API (which any real deployment eventually does, for redundancy or throughput), each instance has its *own* separate dictionary, so a caller could get 3x their real limit just by landing on 3 different instances. The honest fix for a real deployment is a shared store all instances can see (Redis is the standard choice) — writing this limitation down, instead of pretending an in-memory dict fully solves rate limiting, is itself part of the exercise.

**How do you guarantee no secret ever reaches a log line?** `chat_route`'s Advanced hint already established: never put raw exception text in a client-facing reply. The same discipline applies to logs — an agent's step log or a caught exception's message can, in the wrong situation, contain something sensitive (an API key echoed back in an error message from a provider, for instance). A production system usually adds a small redaction step before anything is logged or saved, not just trusts every log line is automatically safe.

Pieces:

- `from fastapi import BackgroundTasks` — `def run_task(request: TaskRequest, background_tasks: BackgroundTasks):` then `background_tasks.add_task(execute_task, run_id, request.input)`.
- A rate limiter's docstring or comment naming its own limitation directly: `# LIMITATION: in-memory only – resets on restart, and each process/worker has its own separate counts.`
- A tiny `redact(text: str) -> str` helper, run over anything before it's logged or saved, replacing anything that looks like a known secret pattern (or, simpler: never log full exception text from a provider SDK call — log the exception *type* and a short summary instead).

**Difference between Basic, Intermediate, and Advanced:** Basic names the 5 pieces and maps each one straight back to a Practice Exercise you already did. Intermediate explains why a 3-state status matters once a run can fail partway through, and gives the exact tools for request-ID logging and a dictionary-based rate limiter. Advanced asks the 3 questions a real deployment can't avoid — synchronous vs. background execution, the actual, honest limitation of an in-memory rate limiter (and why writing it down is required, not optional), and making sure no secret reaches a log line — which is the real difference between "meets the Requirements" and "would survive being someone else's production system."

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Hint 2 — The plan, and almost the whole thing

### Basic Version

```
db/database.py:
    connect to runs.db
    create table runs (id, input, output, log, status, created_at) if missing

api/schemas.py:
    TaskRequest(BaseModel): input: str
    TaskResponse(BaseModel): id, output, status

api/routes.py:
    route GET /health: return {"status": "ok"}

    route POST /run-task, takes a TaskRequest:
        insert a row with status "started"
        try:
            output = call project 4's task-running function with request.input
            update that row: output, status "completed"
        except:
            update that row: status "failed"
            raise a 500
        return TaskResponse

    route GET /runs/{id}:
        look up the row
        if missing: 404
        return it

api/main.py:
    app = FastAPI()
    include the routes

Dockerfile: same shape as db_failure_and_docker's exercise
```

Here's a simplified, all-in-one-file version to get something running before splitting into the suggested files — `run_project_4_task` below is a stand-in, swap in your real one:

```

import sqlite3
from datetime import datetime, timezone
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI()
conn = sqlite3.connect("runs.db", check_same_thread=False)
conn.row_factory = sqlite3.Row
conn.execute("""
    CREATE TABLE IF NOT EXISTS runs (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        input TEXT, output TEXT, log TEXT, status TEXT, created_at TEXT
    )
""")

class TaskRequest(BaseModel):
    input: str

def run_project_4_task(task_input: str) -> str:
    # stand-in for Project 4's real multi-agent entry point
    return "result for: " + task_input

@app.get("/health")
def health():
    return {"status": "ok"}

@app.post("/run-task")
def run_task(request: TaskRequest):
    now = datetime.now(timezone.utc).isoformat()
    cursor = conn.execute(
        "INSERT INTO runs (input, status, created_at) VALUES (?, ?, ?)",
        (request.input, "started", now),
    )
    conn.commit()
    run_id = cursor.lastrowid
    try:
        output = run_project_4_task(request.input)
        conn.execute("UPDATE runs SET output = ?, status = 'completed' WHERE id = ?", (run_id,))
        conn.commit()
        return {"id": run_id, "output": output, "status": "completed"}
    except Exception:
        conn.execute("UPDATE runs SET status = 'failed' WHERE id = ?", (run_id,))
        conn.commit()
        raise HTTPException(status_code=500, detail="Something went wrong.")

@app.get("/runs/{run_id}")
def get_run(run_id: int):

```

```

row = conn.execute("SELECT * FROM runs WHERE id = ?", (run_id,)).fetchone()
if row is None:
    raise HTTPException(status_code=404, detail="run not found")
return dict(row)

```

Run it, `POST /run-task` a couple of times, then `GET /runs/1` and confirm the saved row matches. Rate limiting and per-request log tags aren't in this version yet — that's Intermediate and Advanced.

## Intermediate Version

same as Basic, plus:

middleware: generate a short `request_id` per request, store it on `request.state`, add it to the response headers, and include it in every log line

logging: configure a logger with a formatter that includes `%(request_id)s` (or pass `extra={"request_id": ...}` on every call)

rate limiting: a dict of `{caller_key: [timestamp, timestamp, ...]}`  
 on each request: drop timestamps older than the window, then:  
     if `len(remaining timestamps) >= limit`: raise 429  
     else: append this request's timestamp, proceed

split into the suggested files:

`api/main.py`, `api/routes.py`, `api/schemas.py`  
`db/database.py`, `db/models.py`

Here's most of the middleware and rate limiter, wired into the Basic version's route — fill in the request-ID logging calls yourself:

```

import logging
import time
import uuid
from fastapi import FastAPI, HTTPException, Request

logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(request_id)s] %(message)s")
logger = logging.getLogger(__name__)

RATE_LIMIT = 5          # max requests
RATE_WINDOW = 60        # per this many seconds
request_counts: dict[str, list[float]] = {}

def check_rate_limit(caller_key: str) -> None:
    now = time.time()

```

```

recent = [t for t in request_counts.get(caller_key, []) if now - t < RATE_WINDOW]
if len(recent) >= RATE_LIMIT:
    raise HTTPException(status_code=429, detail="Rate limit exceeded")
recent.append(now)
request_counts[caller_key] = recent

app = FastAPI()

@app.middleware("http")
async def add_request_id(request: Request, call_next):
    request_id = str(uuid.uuid4())[:8]
    request.state.request_id = request_id
    response = await call_next(request)
    response.headers["X-Request-ID"] = request_id
    return response

@app.post("/run-task")
def run_task(request: "TaskRequest", http_request: Request):
    check_rate_limit(http_request.client.host)
    request_id = http_request.state.request_id
    # your turn: pass extra={"request_id": request_id} on every logger.info/.exception
    # call inside this route, so every log line for this run shares the same ID
    ...

```

**Expected output for the 6th request within 60 seconds from the same caller:**

```
429 {"detail": "Rate limit exceeded"}
```

## Advanced Version

same as Intermediate, plus:

```

run the task in the background instead of blocking the whole request:
POST /run-task returns immediately with {"id": ..., "status": "started"}
the actual task execution happens via BackgroundTasks, updating the row
when it finishes (completed or failed)
client polls GET /runs/{id} until status is no longer "started"

```

document the rate limiter's real limitation directly in a comment

```

redact anything log-worthy before it's written, instead of logging raw
exception text from a provider SDK call

```

Here's most of the background-task version — fill in the redaction call yourself:



```

from fastapi import BackgroundTasks, FastAPI, HTTPException, Request

def redact(text: str) -> str:
    # your turn: replace anything that looks like a secret (an API key pattern,
    # for instance) with "[redacted]" before this text is ever logged or saved
    ...
    return text

def execute_task_in_background(run_id: int, task_input: str, request_id: str) ->
    try:
        output = run_project_4_task(task_input)
        conn.execute("UPDATE runs SET output = ?, status = 'completed' WHERE id = ?")
        conn.commit()
        logger.info(f"run {run_id} completed", extra={"request_id": request_id})
    except Exception as exc:
        conn.execute("UPDATE runs SET status = 'failed' WHERE id = ?", (run_id,))
        conn.commit()
        logger.error(f"run {run_id} failed: {redact(str(exc))}", extra={"request_id": request_id})

@app.post("/run-task")
def run_task(request: "TaskRequest", http_request: Request, background_tasks: BackgroundTasks):
    check_rate_limit(http_request.client.host)
    request_id = http_request.state.request_id
    cursor = conn.execute(
        "INSERT INTO runs (input, status, created_at) VALUES (?, 'started', datetime('now'))"
    )
    conn.commit()
    run_id = cursor.lastrowid
    background_tasks.add_task(execute_task_in_background, run_id, request.input, request_id)
    return {"id": run_id, "status": "started"}

```

Try it: `POST /run-task`, note the `id` in the immediate `"started"` reply, then `GET /runs/{id}` repeatedly until `status` becomes `"completed"`.

**Difference between Basic, Intermediate, and Advanced:** Basic runs the task synchronously and saves a 3-state row, all in one file. Intermediate adds real per-request log tagging via middleware and a working (if limited) in-memory rate limiter, split across the suggested files. Advanced questions whether blocking the request for the whole task duration is even right, moves execution to `BackgroundTasks` with client-side polling, and adds the redaction step that keeps a caught exception's raw text out of both logs and the database.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Solution

`run_project_4_task()` below stands in for Project 4's real multi-agent entry point — everything around it (routing, storage, logging, rate limiting, Docker) is what this Build Task is actually about, and doesn't change once you swap the real function in. Read all three depths — they're not "wrong, less wrong, right," they're 3 real, valid ways to meet the same Requirements, with real tradeoffs between them.

### Basic Version

#### Approach 1 — one file, synchronous, meets every stated Requirement

```
# main.py
import sqlite3
from datetime import datetime, timezone
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI()
conn = sqlite3.connect("runs.db", check_same_thread=False)
conn.row_factory = sqlite3.Row
conn.execute("""
    CREATE TABLE IF NOT EXISTS runs (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        input TEXT, output TEXT, log TEXT, status TEXT, created_at TEXT
    )
""")

class TaskRequest(BaseModel):
    input: str

def run_project_4_task(task_input: str) -> str:
    return "result for: " + task_input

@app.get("/health")
def health():
    return {"status": "ok"}

@app.post("/run-task")
def run_task(request: TaskRequest):
```

```

now = datetime.now(timezone.utc).isoformat()
cursor = conn.execute(
    "INSERT INTO runs (input, status, created_at) VALUES (?, ?, ?)",
    (request.input, "started", now),
)
conn.commit()
run_id = cursor.lastrowid
try:
    output = run_project_4_task(request.input)
    conn.execute("UPDATE runs SET output = ?, status = 'completed' WHERE id = ?")
    conn.commit()
    return {"id": run_id, "output": output, "status": "completed"}
except Exception:
    conn.execute("UPDATE runs SET status = 'failed' WHERE id = ?", (run_id,))
    conn.commit()
    raise HTTPException(status_code=500, detail="Something went wrong.")

@app.get("/runs/{run_id}")
def get_run(run_id: int):
    row = conn.execute("SELECT * FROM runs WHERE id = ?", (run_id,)).fetchone()
    if row is None:
        raise HTTPException(status_code=404, detail="run not found")
    return dict(row)

```

```

FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]

```

**Expected output:**

```

$ curl -X POST localhost:8000/run-task -d '{"input": "summarize this"}'
{"id":1,"output":"result for: summarize this","status":"completed"}
$ curl localhost:8000/runs/1
{"id":1,"input":"summarize this","output":"result for: summarize this","log":null}

```

This satisfies every line in the Requirements list except the two not yet added: per-request log tagging, and rate limiting — both below. It's missing the suggested file split, and the `log` column is never populated yet.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Intermediate Version

**Approach 1** — split into the suggested files, request-ID logging, and a working rate limiter

### db/database.py

```
import sqlite3

def get_connection():
    conn = sqlite3.connect("runs.db", check_same_thread=False)
    conn.row_factory = sqlite3.Row
    conn.execute("""
        CREATE TABLE IF NOT EXISTS runs (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            input TEXT, output TEXT, log TEXT, status TEXT, created_at TEXT
        )
    """)
    return conn
```

### db/models.py

```
from pydantic import BaseModel
from typing import Optional

class Run(BaseModel):
    id: int
    input: str
    output: Optional[str]
    log: Optional[str]
    status: str
    created_at: str
```

### api/schemas.py

```
from pydantic import BaseModel

class TaskRequest(BaseModel):
    input: str

class TaskResponse(BaseModel):
```

```

id: int
output: str
status: str

```

### api/routes.py

```

import logging
import time
from datetime import datetime, timezone
from fastapi import APIRouter, HTTPException, Request
from api.schemas import TaskRequest, TaskResponse
from db.database import get_connection
from db.models import Run

logger = logging.getLogger(__name__)
router = APIRouter()
conn = get_connection()

RATE_LIMIT = 5
RATE_WINDOW = 60
request_counts: dict[str, list[float]] = {}
# LIMITATION: in-memory only. Resets on restart, and each worker/instance
# keeps its own separate counts, so real limits under multiple workers or
# instances are effectively RATE_LIMIT * number_of_workers, not RATE_LIMIT.

def check_rate_limit(caller_key: str) -> None:
    now = time.time()
    recent = [t for t in request_counts.get(caller_key, []) if now - t < RATE_WINDOW]
    if len(recent) >= RATE_LIMIT:
        raise HTTPException(status_code=429, detail="Rate limit exceeded")
    recent.append(now)
    request_counts[caller_key] = recent

def run_project_4_task(task_input: str) -> str:
    return "result for: " + task_input

@router.get("/health")
def health():
    return {"status": "ok"}

@router.post("/run-task", response_model=TaskResponse)
def run_task(request: TaskRequest, http_request: Request):
    check_rate_limit(http_request.client.host)
    request_id = http_request.state.request_id

    now = datetime.now(timezone.utc).isoformat()

```

```

        cursor = conn.execute(
            "INSERT INTO runs (input, status, created_at) VALUES (?, ?, ?)",
            (request.input, "started", now),
        )
        conn.commit()
        run_id = cursor.lastrowid
        logger.info(f"run {run_id} started", extra={"request_id": request_id})

    try:
        output = run_project_4_task(request.input)
        conn.execute("UPDATE runs SET output = ?, status = 'completed' WHERE id = ?")
        conn.commit()
        logger.info(f"run {run_id} completed", extra={"request_id": request_id})
        return TaskResponse(id=run_id, output=output, status="completed")
    except Exception:
        conn.execute("UPDATE runs SET status = 'failed' WHERE id = ?", (run_id,))
        conn.commit()
        logger.exception(f"run {run_id} failed", extra={"request_id": request_id})
        raise HTTPException(status_code=500, detail="Something went wrong.")

@router.get("/runs/{run_id}", response_model=Run)
def get_run(run_id: int):
    row = conn.execute("SELECT * FROM runs WHERE id = ?", (run_id,)).fetchone()
    if row is None:
        raise HTTPException(status_code=404, detail="run not found")
    return dict(row)

```

#### api/main.py

```

import logging
import uuid
from fastapi import FastAPI, Request
from api.routes import router

logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(request_id)s] %(message)s")

app = FastAPI()
app.include_router(router)

@app.middleware("http")
async def add_request_id(request: Request, call_next):
    request_id = str(uuid.uuid4())[:8]
    request.state.request_id = request_id
    response = await call_next(request)
    response.headers["X-Request-ID"] = request_id
    return response

```

**Expected output:**

```
$ curl -i -X POST localhost:8000/run-task -d '{"input": "summarize this"}'
HTTP/1.1 200 OK
X-Request-ID: a1b2c3d4
{"id":1,"output":"result for: summarize this","status":"completed"}
```

```
$ for i in 1 2 3 4 5 6; do curl -s -o /dev/null -w "%{http_code}\n" -X POST local
200
200
200
200
200
429
```

The 6th request within the window is rejected, and every server log line for one request shares the same `request_id` shown in its `X-Request-ID` header.

**Difference from Basic:** Approach 1 splits the single file into the suggested `api/` and `db/` layout, adds real per-request log tagging via middleware (every log line for one run now carries the same short ID), and adds a working — if honestly limited — rate limiter, with its limitation written directly into the code as a comment, not left implicit.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Advanced Version

### Approach 1 — background execution with polling, plus log redaction

**api/routes.py** (replacing the synchronous `run_task` from Intermediate)

```
import logging
import re
import time
from datetime import datetime, timezone
from fastapi import APIRouter, BackgroundTasks, HTTPException, Request
from api.schemas import TaskRequest
from db.database import get_connection
from db.models import Run

logger = logging.getLogger(__name__)
router = APIRouter()
conn = get_connection()

RATE_LIMIT = 5
RATE_WINDOW = 60
request_counts: dict[str, list[float]] = {}
# LIMITATION: in-memory only. Resets on restart, and each worker/instance
# keeps its own separate counts – a real multi-instance deployment needs a
# shared store (e.g. Redis) for this limit to mean what it says.

def check_rate_limit(caller_key: str) -> None:
    now = time.time()
    recent = [t for t in request_counts.get(caller_key, []) if now - t < RATE_WINDOW]
    if len(recent) >= RATE_LIMIT:
        raise HTTPException(status_code=429, detail="Rate limit exceeded")
    recent.append(now)
    request_counts[caller_key] = recent

_SECRET_PATTERN = re.compile(r"(sk-[A-Za-z0-9]{10,}|api[_-]?key\s*[:=]\s*\S+)", re.IGNORECASE)

def redact(text: str) -> str:
    return _SECRET_PATTERN.sub("[redacted]", text)

def run_project_4_task(task_input: str) -> str:
    return "result for: " + task_input
```



```

def execute_task_in_background(run_id: int, task_input: str, request_id: str) ->
    try:
        output = run_project_4_task(task_input)
        conn.execute("UPDATE runs SET output = ?, status = 'completed' WHERE id =
            conn.commit()
        logger.info(f"run {run_id} completed", extra={"request_id": request_id})
    except Exception as exc:
        conn.execute("UPDATE runs SET status = 'failed' WHERE id = ?", (run_id,))
        conn.commit()
        logger.error(f"run {run_id} failed: {redact(str(exc))}", extra={"request_

@router.get("/health")
def health():
    return {"status": "ok"}

@router.post("/run-task")
def run_task(request: TaskRequest, http_request: Request, background_tasks: Backg
    check_rate_limit(http_request.client.host)
    request_id = http_request.state.request_id

    now = datetime.now(timezone.utc).isoformat()
    cursor = conn.execute(
        "INSERT INTO runs (input, status, created_at) VALUES (?, ?, ?)",
        (request.input, "started", now),
    )
    conn.commit()
    run_id = cursor.lastrowid
    logger.info(f"run {run_id} started", extra={"request_id": request_id})

    background_tasks.add_task(execute_task_in_background, run_id, request.input,
    return {"id": run_id, "status": "started"}

@router.get("/runs/{run_id}", response_model=Run)
def get_run(run_id: int):
    row = conn.execute("SELECT * FROM runs WHERE id = ?", (run_id,)).fetchone()
    if row is None:
        raise HTTPException(status_code=404, detail="run not found")
    return dict(row)

```

**Expected output:**

```

$ curl -X POST localhost:8000/run-task -d '{"input": "summarize this"}'
{"id":1,"status":"started"}
$ curl localhost:8000/runs/1
{"id":1,...,"status":"started"}
$ sleep 1

```

```
$ curl localhost:8000/runs/1
{"id":1,"input":"summarize this","output":"result for: summarize this","log":null}
```

The client gets an immediate reply instead of waiting for the full task, and polls `GET /runs/{id}` until `status` moves past `"started"` — the shape that matters once a real multi-agent task takes long enough that blocking the HTTP request for it would be a mistake.

## Approach 2 — Dockerfile + `.dockerignore` + required-env-var check, and `test_api.py`

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["uvicorn", "api.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

```
# .dockerignore
.env
.git
__pycache__/
*.pyc
.venv/
*.db
```

```
# api/main.py – add this near the top, before app = FastAPI()
import os
OPENAI_API_KEY = os.environ["OPENAI_API_KEY"] # fails the container immediately
```

```
# test_api.py
from fastapi.testclient import TestClient
from api.main import app

client = TestClient(app)

def test_health():
    assert client.get("/health").status_code == 200

def test_run_task_and_lookup():
    response = client.post("/run-task", json={"input": "hello"})
    assert response.status_code == 200
    run_id = response.json()["id"]
    lookup = client.get(f"/runs/{run_id}")
    assert lookup.status_code == 200
```

```
def test_bad_request_returns_422_and_writes_nothing():
    response = client.post("/run-task", json={})
    assert response.status_code == 422

def test_rate_limit_returns_429_eventually():
    statuses = [client.post("/run-task", json={"input": "x"}).status_code for _ in range(10)]
    assert 429 in statuses
```

**Expected output:**

```
$ docker build -t project-5-api .
$ docker run -p 8000:8000 -e OPENAI_API_KEY=sk-test-123 project-5-api
$ curl localhost:8000/health
{"status":"ok"}
$ docker history --no-trunc project-5-api | grep -i "sk-test-123"
(no output)

$ pytest test_api.py
4 passed
```

**Difference from Intermediate, and between these 2 Advanced approaches:** Intermediate's synchronous route already meets every stated Requirement, but blocks the whole HTTP request for as long as the multi-agent task takes. Approach 1 changes the execution model — respond immediately, run the task via `BackgroundTasks`, poll for the result — and adds the `redact()` step so a caught exception's raw text (which could echo back something sensitive from a provider SDK) never reaches a log line. Approach 2 doesn't touch route logic at all — it closes out the operational requirements: a Dockerfile and `.dockerignore` that keep secrets out of every image layer, a required-env-var check that fails the container loudly at startup, and a real `test_api.py` covering the 5 scenarios from this Build Task's own Test Cases table.

**Which one should you actually write?** Intermediate Approach 1 already satisfies every line in the Requirements section, and is what most people should ship first — get the whole loop (route → database → logs tagged by request → Docker → rate limit) working synchronously before making it more complex. Move to Advanced Approach 1's background execution once you notice `run_project_4_task()` genuinely takes long enough that a blocked HTTP request becomes its own problem (a client-side timeout, a tied-up worker) — which, per this document's Goal, is exactly what happens once you plug in the real Project 4 multi-agent system instead of the stand-in used here. Do Advanced Approach 2's Dockerfile, `.dockerignore`, required-env-var check, and `test_api.py` regardless of which execution model you chose — none of it is optional for something you'd actually deploy, and it's exactly what [14\\_debugging\\_lab](#)'s Break-It drills will test against.